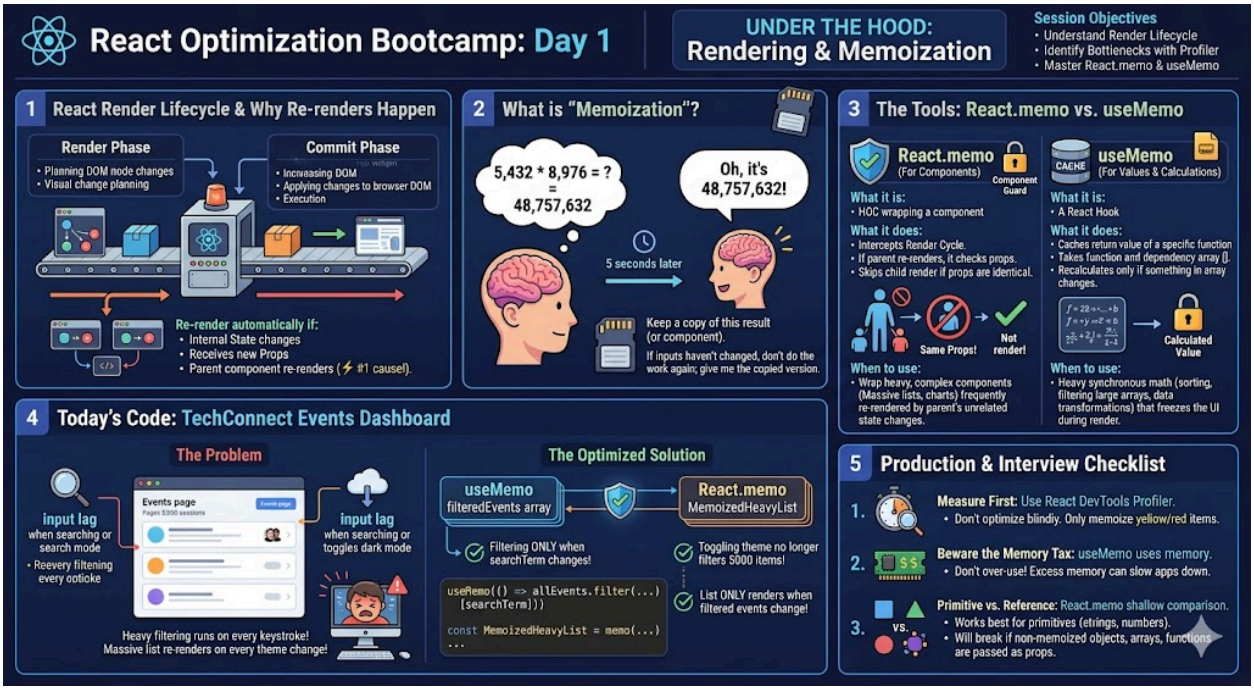
Day 1: Conquering the Re-render (`React.memo` & `useMemo`)

🎯 Objectives

- Understand the React Render Lifecycle (Render vs. Commit phase).
- Learn how to identify performance bottlenecks using the **React DevTools Profiler**.
- Master `React.memo` to prevent child components from re-rendering uselessly.
- Master `useMemo` to cache expensive, heavy calculations.



🧠 The "Why" Behind Re-renders

React is fundamentally reactive. A component will automatically re-render if:

1. Its internal `state` changes.
2. It receives new `props` (or the reference to those props changes).

3. **Its parent component re-renders.** *(This is the #1 cause of performance degradation in large apps!)*

💡 What exactly is "Memoization"?

Memoization is just a fancy programming term for **caching**.

Imagine someone asks you, *"What is 5,432 multiplied by 8,976?"* It might take you a few minutes to calculate the answer (48,757,632). If they ask you the exact same question five seconds later, you don't calculate it again—you just remember the answer.

In React, memoization means telling the framework: *"Keep a copy of this result (or this component). If the inputs haven't changed, don't do the hard work again; just give me the copied version."*

🛠️ The Tools: `React.memo` vs. `useMemo`

It is easy to confuse these two, but they serve different purposes:

1. `React.memo` (For Components)

- **What it is:** A Higher-Order Component (HOC) that wraps an entire component.
- **What it does:** It intercepts the render cycle. If a parent component re-renders, `React.memo` checks the props of the child. If the props are *exactly the same* as the last render, it skips rendering the child entirely.
- **When to use it:** Wrap heavy, complex components (like massive lists, data tables, or charts) that are frequently forced to re-render by a parent's unrelated state changes.

2. `useMemo` (For Values & Calculations)

- **What it is:** A React Hook.
- **What it does:** It caches the *return value* of a specific function. It takes two arguments: the function doing the work, and a dependency array `[]`. It only recalculates the value if something inside that array changes.
- **When to use it:** When you are doing heavy synchronous math (sorting 10,000 rows, filtering massive arrays, data transformations) that freezes the UI during

a render.

💻 TechConnect Events Dashboard

Today, we started building **TechConnect**, our large-scale conference management portal.

The Problem: Our Events page loads 5,000 sessions. Initially, typing in the search bar or toggling the Dark Mode theme caused massive input lag. Why? Because the heavy filtering array ran on *every single keystroke*, and the massive list re-rendered even when just the theme changed.

The Optimized Solution

Here is the final, optimized code from today's session. We used `useMemo` to cache the search results and `React.memo` to shield the massive list from the parent's state changes.

JavaScript

```
import React, { useState, useMemo, memo } from 'react';

// Simulated large dataset: 5000 Conference Events
const allEvents = Array.from({ length: 5000 }, (_, i) => ({
  id: i,
  title: `Global Tech Summit ${i}`,
  speaker: `Speaker ${i}`,
}));

export default function EventsDay1() {
  const [searchTerm, setSearchTerm] = useState('');
  const [isDarkMode, setIsDarkMode] = useState(false);

  //  OPTIMIZATION 1: useMemo
  // This caches the filtered array. It ONLY recalculates when 'searchTerm' changes.
  // Toggling the theme no longer triggers this heavy 5000-item filter!
  const filteredEvents = useMemo(() => {
    console.log("Filtering 5000 items... 🧠");
    return allEvents.filter(event =>
      event.title.toLowerCase().includes(searchTerm.toLowerCase())
    );
  }, [searchTerm]);

  return (
    <div className={`p-8 rounded-2xl shadow-sm border transition-colors ${isDarkMode ? 'bg-
slate-900 border-slate-800 text-white' : 'bg-white border-slate-200 text-slate-900'}`}>
      <div className="flex justify-between items-center mb-6">
        <h2 className="text-2xl font-bold">Event Schedule (Day 1)</h2>
```

```
        <button
          onClick={() => setIsDarkMode(!isDarkMode)}
          className="px-4 py-2 bg-indigo-100 text-indigo-700 font-semibold rounded-md
hover:bg-indigo-200"
        >
          Toggle Theme
        </button>
      </div>

      <input
        type="text"
        placeholder="Search thousands of events..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        className="w-full p-4 rounded-xl border border-slate-300 text-black shadow-sm
focus:ring-2 focus:ring-indigo-500 outline-none mb-6"
      />

      {/* OPTIMIZATION 2: React.memo */}
      {/* This list will now ONLY re-render if the 'filteredEvents' array actually changes.
*/}
      <MemoizedHeavyList events={filteredEvents} />
    </div>
  );
}

// React.memo wraps the child component
const MemoizedHeavyList = memo(({ events }) => {
  console.log("Rendering Heavy List... ⚡");
  return (
    <div className="border border-slate-200 rounded-xl overflow-hidden bg-slate-50">
      <ul className="h-[400px] overflow-y-auto divide-y divide-slate-200">
        {events.map(event => (
          <li key={event.id} className="p-4 hover:bg-indigo-50 flex justify-between text-
slate-800">
            <span className="font-semibold">{event.title}</span>
            <span className="text-slate-500 text-sm">{event.speaker}</span>
          </li>
        ))}
      </ul>
    </div>
  );
});
```

🚀 Production & Interview Checklist

When working in enterprise codebases (or answering system design questions in an interview), keep these rules in mind:

- **Measure First:** Never optimize blindly. Use the React DevTools Profiler. If it doesn't show up as yellow/red in the flamegraph, don't memoize it.
- **Beware the Memory Tax:** `useMemo` uses memory to store the cached result. Wrapping every single variable in `useMemo` will actually make your app slower and consume more RAM.
- **Primitive vs. Reference:** `React.memo` works via a shallow comparison. It works great for primitives (strings, numbers, booleans). But if you pass an un-memoized object, array, or function as a prop, `React.memo` will break.